Field Hours

CSCE 606 Software Engineering

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 40 minutes

A spec, a test, and a method

Software engineering in this course is the set of practices that change the rate of defects and the cost of a change. A methodology is a choice of which artifacts you keep: a written spec, tests that fail for a named reason, a review, a build that someone else can run. Collecting the artifacts without using them is not a methodology. It is a shelf. Pick the ones that catch the bugs you actually have, and drop a ceremony that has not caught a bug in a month.

A specification says what the function must do on inputs you care about, including the inputs that should be rejected. 'Sort the list' is not a spec until you say whether duplicates stay, whether the input is preserved, and what an empty list returns. A test is a spec the machine can run. The test should fail for a reason you can point at. A test that passes before the code is written, or that cannot fail, is not a test. Delete it.

Prototyping is a method, not a lack of one. A prototype exists to answer a question cheaply: will a person understand this control, does this file format hold the records, is the query fast enough on a fake million rows. The prototype is thrown away, or it is not a prototype. The failure mode is shipping it. Write the question on the first line of the prototype's notes. When the question is answered, stop.

Reuse is a method with a cost. A library you adopt is a defect surface you did not write and a change schedule you do not control. Reuse it when the spec matches and you can test the boundary. Copying a function you do not understand is not reuse. It is a delayed bug with someone else's variable names.

The order that keeps a method honest. A test that cannot fail never earns a place in the build.

1. Write the behavior in one paragraph
2. Write a test that fails
3. Make it pass
4. Keep the test in the build

A parser for a tiny record

1. Spec: a record is two integers separated by a comma. Extra spaces are rejected. An empty string is rejected.

2. The first test feeds '1, 2' and expects rejection, because of the space. It fails if the code accepts it.

3. The second test feeds '' and expects rejection. A parser that returns zeros for empty input fails this test, which is the point.

Problem 1. A test calls the function and does not check the return value. The build is green. What do you do with the test?

Answer. Remove it or make it assert a result. A test that cannot fail will stay green while the function breaks. It is not evidence.

Problem 2. You keep a prototype because the demo went well. What question did you fail to answer before shipping it?

Answer. Whether it meets the spec you would write for a system you have to change: errors, tests, and the cases the demo did not click. A prototype answers a narrow question. Shipping it means you skipped the spec.

Lecture 2 · 40 minutes

Change that stays local

A module is a boundary with a small interface and a secret behind it. The secret can change. Callers cannot. If changing the representation of a date forces edits in forty files, the date had no module. The interface is the set of operations, not a class keyword. A module that exposes its list so callers can append is a module in name only.

Design for change means naming the changes you expect. A file format you expect to extend gets a version field. A tax rule you expect to vary becomes a function you can replace, not a constant in twenty formulas. You will guess wrong about some changes. The exercise is still useful, because an unnamed change is sprayed through the program, and a named one is at least in one place when it arrives.

Reuse inside a program is the same boundary. Two callers that need the same rule should call one function. Two callers that need almost the same rule are a trap: a boolean flag that switches behavior makes both callers pay for both behaviors. Prefer two small functions, or a parameter that is a real part of the spec, not a mode you hope nobody combines.

Coupling is a path by which a change spreads. A global configuration object that every file reads is a path. A cycle of imports is a path. You do not need a metric to see it. Try the change on paper. Count the files. If the count embarrasses you, the boundary is in the wrong place.

One change, two designs. The right-hand design is what an interface is for.

No boundary

The list is public

Forty files append

A new rule edits all of them

A small interface

Callers ask for a result

The store stays hidden

The rule changes in one file

A rate that will change

1. Three reports multiply hours by 20. The 20 is the hourly rate, copied.

2. The rate becomes 25. You must find all three, and you will miss a fourth in a test that used 20 as an anonymous expected value.

3. A function rate_for(job) puts the 20 in one place. The reports call it. The test calls it or states 25 times hours as a derived expectation.

Problem 1. A function takes a boolean called simple_mode that skips validation. Why is that a bad parameter?

Answer. Callers can combine it with paths that assumed validation ran. The spec is now two specs, and the type does not say which one you are in. A separate function with its own name makes the skipped checks visible at the call.

Problem 2. You expect the on-disk record to grow a field next year. What do you put in the format now?

Answer. A version number, and a parser that rejects versions it does not know. Without the version, old and new bytes are ambiguous and every reader grows a guess.

Lecture 3 · 40 minutes

Invariants and review

An invariant is a fact that is true every time control passes a point you named. A stack's invariant is that the size is the number of items and is never negative. A function may break the invariant in the middle of its body and must restore it before it returns or throws. State the invariant next to the data. A bug is often an invariant you believed and did not check.

Assertions check invariants in the build you test. They are not error handling for a user. An assertion that fails means the program is wrong. An error code means the user asked for something the spec rejects. Mixing them hides defects inside a polite message. Keep both, and know which one fired.

Review is another person reading the change against the spec and the invariant. The useful comment is 'this path leaves the size negative' or 'this test does not cover an empty input'. The useless comment is taste about names, unless the name lies. A review that only checks style will approve a broken invariant that was neatly indented.

Name the defect you are hunting. Off-by-one, a missing unlock, a retry that applies twice, a test that cannot fail. A review checklist of twenty generic items is a way to not look at the diff. Three questions aimed at this module catch more: what is the invariant, which test fails if it breaks, who else must change.

An invariant is a door. You may break it inside the room. You may not leave through the door without it.

1. State the invariant

2. Break it only inside the function

3. Restore it before return

4. Assert it in tests

A stack

1. Invariant: size equals the number of stored items, and size is at least 0.

2. Pop on an empty stack that returns a default and decrements size leaves size at -1. The invariant is false.

3. The repair is to refuse the pop when size is 0, as an error, and to assert size is at least 0 at the end of every public function.

Problem 1. A reviewer says the variable should be called count instead of n, and misses that the lock is not released on the error path. What did the review measure?

Answer. Style, not the invariant. The missing unlock is the defect. A review checklist that starts with names will keep producing this result. Ask where the lock is released on every return.

Problem 2. An assertion fires on a user's bad input. Why is the assertion the wrong tool?

Answer. The input is in the spec, as a rejection. The program should return an error the caller can handle. An assertion says the programmer is wrong, and it may be compiled out of the build the user runs.

Lecture 4 · 40 minutes

History, builds, and environments

Version history is how you know what you shipped. A commit should build, and the message should say why the change exists. A history of 'fix' and 'more' cannot be bisected. Bisect is the tool that finds the commit which broke a test, and it works only if the old commits build. A commit that does not build taxes every future search.

The build is the environment. 'Works on my machine' means the machine has a compiler, a library, or a file the build did not name. The repair is to name them. A lockfile or a vendored dependency, a script that creates the database, a documented compiler version. A new person should run one command and get the tests. If they cannot, you do not have a build. You have a workstation.

Environments multiply: a laptop, a test box, a production box. Configuration that differs belongs in the environment, not in a fork of the code. A fork will miss the next bugfix. The same binary, different configuration, is the rule. Secrets do not belong in the history. A key committed once is a key you must rotate, because history is copied.

Reproducibility is a engineering property with a test. Two people on two days should get the same test result from the same commit. If they do not, the environment is part of the program and it is untested. Treat a flaky test as a defect in that property, not as a test to rerun until it passes.

The commit names the code. The build names the rest. A laptop is not a layer.

One command runs the tests

Pinned dependencies

A commit that builds

The change and why

A library that moved

1. The code imports a library and does not pin a version. This month the tests pass.

2. Next month the library changes a default. The tests fail on a new laptop and pass on the old one, which still has the old install.

3. Pinning the version, or vendoring it, makes the failure a deliberate upgrade instead of a ghost.

Problem 1. You need a database password to run the tests. Where does it go?

Answer. In the environment of the machine running the tests, or in a local file the history ignores. Not in a commit. If it was committed, rotate the password. Deleting the file in a later commit leaves it in the old one.

Problem 2. Bisect stops on a commit from last year that does not compile. What does that cost you?

Answer. You cannot use history to find which change broke the test, at least not past that commit. The old broken commit is a defect in the history. The repair going forward is to keep the main line buildable.

Lecture 5 · 45 minutes

A change you can defend

The last exercise is a maintenance change on a small program you specify yourself. Write a twenty-line program that parses the two-integer record from the first lecture and returns the sum, rejecting bad input. Put the tests in the build. Then change the spec: a record may have a third optional integer, and the sum includes it when it is present. Do the change without breaking the old tests.

The old tests are the contract with yesterday. If you edit an old test so the new code passes, write down why the spec changed. An edited test that quietly expects the new behavior is a deleted contract. Add new tests for the third integer. Keep the rejection tests for spaces and empty strings unless the spec says those are now legal.

Prototyping belongs in the middle of the change if you are unsure. A throwaway parser that only answers 'can a third field be optional without eating the error cases' is legitimate. The code you commit is the one the tests cover. The prototype does not get a special exemption because the change was small.

Write the review notes as if you were the second person. Where is the invariant? Which test fails if the optional field is required by mistake? Which files changed? If the answer is 'the parser and its tests', the boundary held. If the answer is 'the parser, the reports, and three copies of the split', you skipped lecture two, and the change is not done.

A maintenance change. Old tests stay. New tests arrive with the new clause of the spec.

Old tests green -> Change the spec -> Add tests -> Then the code

The optional third integer

1. Before: '1,2' sums to 3. '1,2,3' is rejected. '' is rejected.

2. After: '1,2' still sums to 3. '1,2,3' sums to 6. '1,2,' is rejected because the third field is empty, not absent.

3. A single split that treats a trailing comma as a zero fails the new rejection, and it should.

Problem 1. '1,2' used to return 3 and now returns 6 because you always add a hidden default. Which

test catches it?

Answer. The old test that '1,2' sums to 3. If you updated that test to expect 6, you erased the spec. The third field is optional, so its absence must leave the old sum.

Problem 2. The change touched the parser, the report formatter, and a copy of the parser in a script. What should you do before you call it done?

Answer. Put the parse in one module and make the script and the formatter call it. Three copies will diverge on the next change. The boundary is the unfinished part of the work.

Lecture 6 · 180 minutes

Requirements you can fail

A requirement is a sentence a test can reject. 'The system should be fast' is not a requirement. 'A recorded payment is visible to a second session within one second, or the user is shown a pending state' is a requirement. The earlier lecture asked for a spec and a test. This week is how to write the sentence so the test is obvious.

Use cases are the sequences you will actually support, including the failures. A transfer that debits and then loses the network is a use case, not an edge you will get to later. If it is not in the list, the implementation will invent a behavior and the invention will ship.

Non-functional requirements need a number and a condition. 'Available' needs a definition of downtime and a window. 'Secure' needs a threat, which the security course will demand anyway. Import the threat. Do not write the word secure.

Trace each requirement to a test name. A requirement with no test is a wish. A test with no requirement is a guess about what matters. The matrix can be a list of pairs. It does not need a tool.

A wish and a requirement.

Not testable

Fast

User friendly

Secure

Testable

Under 200 ms at the stated load

A new user finishes the task in two minutes

The lost-response retry does not double-charge

One payment sentence

1. Wish: payments should not be lost or duplicated.

2. Requirement: if the client retries with the same request id after a timeout, the server returns the original result and commits at most one charge.

3. The test is the late-acknowledgement case from the networks course, applied to this spec. You can fail it on purpose by deleting the id check.

Problem 1. Rewrite 'the page should load quickly' as a requirement a test can fail.

Answer. State the page, the network condition, and a number. For example: on a cold cache and a 50-millisecond round-trip, the payment page's HTML is complete in under 300 milliseconds for the fixture user. A test that measures 900 milliseconds fails the requirement.

Problem 2. A test exists for a behavior no requirement mentions. What do you do with it?

Answer. Either write the requirement it is protecting, or delete the test. An unexplained test becomes a reason to keep an accidental behavior forever. The trace goes both ways.

Lecture 7 · 180 minutes

Modules

A module is a boundary with a responsibility and a secret. The secret is what other modules may not depend on: a data layout, a protocol, a vendor. The earlier lecture on local change is this boundary stated as a test. If a change to the secret forces edits outside the module, the boundary is a comment, not a module.

The dependency arrow points toward the thing whose changes you want to absorb. A domain module that imports a web framework will change when the framework changes. Invert it: the web module depends on the domain, and the domain does not mention HTTP. The arrow is a design decision you can check with a search.

A layer that every call passes through, and that contains no policy, is a pass-through. Delete it or give it a secret worth hiding. Layers are not a virtue. Each one must earn the indirection by absorbing a change you can name.

Interfaces are small. A module that exposes its map of records has no secret. Expose the operations the use cases need. When a new use case needs a new operation, that is a real change to the boundary, and it should be visible in review.

Arrows that may point inward.

HTTP handlers depend on the domain

The domain does not import the handlers

The store is a secret of one module

A change to the table stays in that module

A rate that will change

1. The earlier example hid a rate behind a function. Put that function in one module.

2. Search the code for the numeric rate. It appears in that module and in tests of that module. It does not appear in the handler.

3. If it appears in the handler, the boundary failed. Move it. Do not add a comment telling people not to duplicate it.

Problem 1. The domain module imports the HTTP request type so it can 'see the user.' What change

will ripple?

Answer. A change of web framework, or of the request object, will edit the domain. Pass the user id or a domain object in. The domain should not mention the framework.

Problem 2. A module's public function returns the internal dictionary. Why is that a broken secret?

Answer. Callers can insert keys the module does not validate, and they depend on the dictionary type. You cannot change the representation. Return the specific values the caller needs, or an object that does not expose the collection.

Lecture 8 · 180 minutes

Tests that find faults

A test suite can be green and useless. Line coverage says which lines ran, not which faults would be caught. A test that calls a function and never checks the result covers lines and catches nothing. Assertion is the test. The call is the setup.

Boundary values are the cases next to a change in behavior: zero, one, the capacity, the capacity plus one, an empty name, a duplicate id. Pick them from the spec, not from the code's current branches. If you only test the branches the code already has, you will not notice a missing branch.

A regression test is a test you add when you fix a bug, which fails on the old code and passes on the new. It is how the bug stays fixed. A bug fix without a test will return. The lecture on history and builds told you the environment. This is what you put in it.

Flaky tests are tests that depend on time, order, or a network. They train the team to ignore red builds. Quarantine is a temporary state with a ticket, not a home. A suite people rerun until it passes is not a suite.

From a bug to a permanent test.

1. Reproduce it in a test

2. Watch the test fail

3. Fix the code

4. The test stays

An off-by-one capacity

1. A buffer holds 2 items. The spec says a third insert returns an error and leaves the two items in place.

2. A test inserts two, asserts success, inserts a third, asserts the error, and asserts the length is still 2.

3. A buggy implementation that overwrites the first item fails the length assertion. A test that only checks the error code, and not the surviving contents, misses that bug.

Problem 1. Coverage is 100 percent and a bug remains. How is that possible?

Answer. The bug is a missing check, or a wrong check on a value the tests never asserted. Every line ran with inputs that did not expose it. Coverage does not generate the boundary you forgot.

Problem 2. A test passes alone and fails when the suite runs. What kind of bug is that, and where do you look?

Answer. Order dependence: leftover global state, a shared file, a clock. Look at what the previous test mutated. The production code may be fine and the tests may be sharing a world they should not share.

Lecture 9 · 180 minutes

Review

A review is a reading of a change against the spec and the invariants, before it merges. It is not a style referendum and not a test run. The tests already ran. The reviewer looks for the requirement the change violates, the secret that leaked, and the missing failure case.

The author attaches the requirement and the test names. A review of a diff with no statement of intent becomes a review of taste. If you cannot say what the change is for, it is not ready.

Comments in a review are either defects or questions. A defect names the failing input. A question means the reviewer does not understand, which is itself a problem if the code is supposed to be readable. 'Nit' comments on names are allowed after the defects, and they are not a reason to block a correct change forever.

The author does not merge their own change on a module another person can read. Solo work still needs a pause and a checklist. The checklist is the invariant list from the earlier lecture, plus the requirement id.

What a review reads, in order.

The requirement

The tests that should fail without the change

The invariants

Only then the names

A change that forgets the retry

1. The diff adds a charge. The requirement says a retry with the same id charges once. The diff has no id.

2. The reviewer cites the requirement and the missing test, not a preference about method length.

3. The author adds the test first, watches it fail, then adds the id. That order is the review doing its job.

Problem 1. A reviewer asks for a rename and does not mention a missing error case you can see in the diff. What should happen to the merge?

Answer. The missing error case blocks the merge. The rename can happen in the same change or a follow-up. Treating the rename as the review, and merging the missing case, is the failure mode of review-as-style.

Problem 2. Why attach the requirement id to the change?

Answer. So the reviewer can check the right sentence, and so a later reader can find why the code

exists. A diff without an intent can only be checked for local taste and obvious crashes.

Lecture 10 · 180 minutes

Change over time

Software spends more time being changed than being written. The cost of a change is the modules you must open and the tests you must rethink. A design is good if the likely changes are local. You cannot make every change local. You choose which changes are likely and localize those.

Technical debt is a specific shortcut: a missing module boundary, a duplicated policy, a test you skipped, with a named cost to pay later. It is not a synonym for code you dislike. Write the cost down or you will 'repay' debt by rewriting the parts that were fine.

A deprecation needs a time and a caller list. Breaking an interface with no remaining callers is a deletion, not a deprecation. Breaking one with callers you did not find is an outage. Search for the callers as part of the change.

Version the data, not only the code. A record written last year must still load, or you need a migration with a test. The migration is a program. It gets a review.

A change you expected.

1. Name the likely change

2. Hide it in one module

3. When it arrives, edit that module

4. Add the regression test

A second payment provider

1. The likely change, written on day one, is a second provider. The secret of the first provider sits behind an interface with charge(id, amount).

2. The second provider arrives. You add a module. The handlers do not change, if the interface was honest.

3. If the handler built the first provider's JSON, the change touches every handler. That cost is the measure of the earlier shortcut. Write it down as debt if you defer the interface, including the day you will pay it.

Problem 1. Someone calls a messy module 'debt' and proposes a rewrite of the whole service. What is missing?

Answer. The specific shortcut and its cost. Debt is payable because it is scoped. A rewrite of everything is a new project. Ask which change is hard, and localize that.

Problem 2. Old records lack a field the new code requires. What has to ship with the code?

Answer. A migration or a default that the spec accepts, plus a test that loads an old record. Shipping the code alone will fail when it reads yesterday's data.

Lecture 11 · 180 minutes

When it is running

A running system fails in ways a unit test did not arrange. Logs, metrics, and a way to tell which version is running are part of the software. A log line that does not include the request id cannot be tied to the failed payment. A metric that only exists on your laptop does not exist.

An alert needs an action. 'CPU is high' is not an action. 'The payment error rate exceeded one percent over five minutes, and the runbook says to fail over the provider' is an alert. Page a person only for the second kind.

A rollback is a feature. If the migration cannot roll back, the deploy is a one-way door and the review should say so. Practice the rollback before the night you need it. A rollback you have never run is a rumor.

Incidents get a short note: what broke, how you know it is fixed, and what test or alert you added. Blame is not a repair. The software-engineering content of an incident is the missing test.

Operate the change.

A request id in the log

A metric for the requirement

An alert with an action

A rollback you have run

A failed deploy

1. Error rate on charges rises after the deploy. The request ids in the logs match the new version.

2. You roll back. The error rate falls. The version metric confirms the old code is what is running.

3. You add a test for the case the new code mishandled, and you write three sentences. The next deploy does not repeat it by accident.

Problem 1. The only log line is 'something went wrong.' Why can you not operate this?

Answer. You cannot tie the line to a request, a user, or a version, and you cannot count it. Add the request id and the error class. A unique sentence per failure is less useful than a class you can graph.

Problem 2. An alert fires every night and is always ignored. What should you change?

Answer. The alert. Either it is not actionable and should be a dashboard, or the nightly condition is a real bug and the runbook is wrong. An ignored page trains you to ignore the next one.

Lecture 12 · 180 minutes

The maintenance exercise

Take a program you wrote earlier in the degree and change it. The change must be a requirement with a test, reviewed against an invariant, and small enough to finish. The earlier lecture's optional third integer is this exercise in miniature. Here you also write the rollback and the log line.

Before you edit, find the module that should own the change. If the change wants to touch five modules, stop and

ask whether the boundary is wrong or the requirement is five requirements. Split until one of them is local.

Measure the diff. A change of the behavior that adds two hundred unrelated lines is two changes. Split the commits. A reviewer, even if the reviewer is you tomorrow, should see the requirement in one commit.

This is the course's project. The algorithms, the database, and the network code you wrote are eligible targets. Changing them under a spec is the software-engineering skill. Writing a new feature with no spec is a different, weaker exercise.

The exercise.

1. A failing test for the new sentence

2. The smallest module

3. The change

4. A log line and a way back

The optional third integer

1. The spec gains: an optional third integer, defaulting to 0, and the sum includes it.

2. The test calls the old two-argument form and expects the same sum as before, then calls the three-argument form.

3. If the old form breaks, the change was not compatible. The requirement did not ask you to break callers. The default is the compatibility.

Problem 1. The change is correct and the diff reformats the whole file. What do you do before review?

Answer. Split the format from the behavior, or drop the format. A reviewer cannot see the requirement inside a whitespace storm. The behavior commit should be readable as the requirement.

Problem 2. You cannot find a module that can own the change without also editing the HTTP layer and the store. What does that say?

Answer. The policy is smeared across layers. The exercise now includes putting the policy in one place first, then the behavior change. Doing both in one undiffable edit hides the design repair.

Lecture 13 · 180 minutes

Quality is a trade

You cannot maximize every quality. Reliability, time to ship, performance, and a clean boundary compete. The engineering decision names the one you are sacrificing and the one you are protecting. 'We will ship the retry bug because the demo is tomorrow' is an honest, usually bad, decision. Silence about the bug is not a decision.

A prototype may drop the durable log and must not be confused with the system. Mark it. The earlier human-computer course will prototype interfaces. Those prototypes are not the payment path. Throw them away or rewrite them against the requirements.

Reuse is copying a module whose secret still matches, not copying a function you do not understand. A copied function is a second secret that will diverge. The dependency, if you can take it, is better. If you cannot, write

your own and test it.

Artificial intelligence tools that write a patch are junior authors. The requirement, the test, and the review still belong to you. A patch you cannot explain is a patch you cannot maintain. Do not merge it.

Say the trade.

Protect

The single charge

The migration

The rollback

Sacrifice, on purpose

A nicer name

A second provider this week

A perfect boundary you have not needed

A demo tomorrow

1. The retry bug is real. The demo is tomorrow. You can disable retries for the demo and say so, which protects correctness by narrowing the feature.

2. Or you can ship the retry and risk a double charge in front of an audience. That protects the feature list and sacrifices the requirement.

3. Either choice can be written down. Showing the retry and hoping is the choice that cannot.

Problem 1. A prototype's code is the shortest path to the demo and fails the durability requirement. How do you label the branch?

Answer. As a prototype that must not be deployed. The label is in the README and in the absence of the production config. Merging it to the release branch unlabeled is how prototypes become the system.

Problem 2. A generated patch passes the tests and you do not understand the lock it added. Do you merge it?

Answer. No. You can learn it, or you can reject it. The tests did not review the invariant. An unexplained lock is how you add deadlock. The author of record is you.

Lecture 14 · 180 minutes

The engineering bar

The course is a method you can apply to every other course's code. Specify the behavior a test can reject. Hide the likely change. Check the invariant in review. Leave a way to tell, in production, that the requirement is failing, and a way to go back.

You are done with a week when you can apply it to a small program without looking at these notes. The hours

are that practice. Reading the week is the start.

The capstone will be graded, by you, against this bar. A capstone that cannot say what a retry does, where the policy lives, and how you know it is up, is unfinished regardless of how many features it lists.

These notes are original. They are the standard method, not a transcription of a term at a particular school. Use them on your code.

The bar.

A sentence a test can fail

A module that owns the secret

A review against an invariant

A signal in production and a way back

Apply it to a sort

1. Requirement: the function returns a new list, sorted, and does not mutate the input.

2. The test passes a list, keeps a copy, and compares. A sort that sorts in place fails.

3. The likely change is the comparison. It sits in one function. The review checks mutation, not the variable names.

Problem 1. Which part of the bar is missing if the sort is correct, tested, and nobody can tell in the running system whether the endpoint that calls it is failing?

Answer. The production signal. Correctness in a unit test does not operate the service. Add the error metric or the log with a request id on the endpoint. The sort can stay.

Problem 2. Name the likely change for a parser of a record format, and where it should live.

Answer. A new field. It should live in the parser module, with a test that the old format still parses. Callers should see a default, not a new argument they must all learn on the same day, unless the requirement says the field is mandatory.